CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: OrderFlow (Food Ordering App)

Prepared By: Ycoy, Dillan Marquin P.

Version: 1.0

Date: Feb 23, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	[Date]	[Your Name]	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	Feb 23, 2026	Dillan Marquin Ycoy	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

OrderFlow is a web and mobile-based Food Ordering and Transaction Management System designed to manage structured food ordering between Buyers (Customers) and Sellers (Restaurant Administrators).

The system allows buyers to browse food items, manage a shopping cart, complete checkout using a real sandbox payment gateway (Stripe Test Mode), and receive confirmation via email. Sellers manage food listings, upload product images, and process customer orders.

The core focus of OrderFlow is its **Transaction Processing Module**, which ensures:

- Cart validation
- Secure sandbox payment processing
- Order creation after successful payment
- Payment result recording
- Email receipt generation
- Structured order management

The system implements:

- JWT Authentication
- Google OAuth Login
- Role-Based Access Control
- External API Integration
- File Upload
- SMTP Email Sending
- Android Mobile App Integration (Kotlin + XML)

OrderFlow follows a Layered Architecture pattern for clean separation of concerns and maintainability.

1.2 Objectives

The objectives of OrderFlow are:

1. To implement secure authentication using JWT and Google OAuth.
2. To enforce role-based access control between Buyers and Sellers.
3. To design a complete end-to-end transaction workflow.
4. To integrate Stripe Test Mode for sandbox payment processing.
5. To consume a public external API and display meaningful data.
6. To implement file upload functionality.
7. To send real emails using SMTP.
8. To provide a fully functional Android application connected to the same backend.

1.3 Scope

Included Features

- User Registration & Login
- Google OAuth Login
- JWT Authentication
- Protected API endpoints
- Role-based access control
- Food item CRUD (Seller)
- Shopping cart system
- Stripe Test Mode payment integration
- Order creation after successful payment
- Payment result recording

- File upload (food images)
- External API integration (nutrition data)
- SMTP email notifications
- Android mobile app (Kotlin + XML)

Excluded Features

- Real money charging (production mode disabled)
- Live GPS tracking
- Multi-restaurant expansion

1.0 INTRODUCTION

1.1 Purpose

This document defines the complete system design of OrderFlow, including authentication, transaction workflow, payment processing, integrations, and technical architecture.

It serves as the official development reference for Final Defense implementation.

1.2 System Overview

OrderFlow connects two primary roles:

Buyer (Customer)

- Browses menu
- Adds items to cart
- Completes payment (Stripe Test Mode)
- Receives email receipt
- Views order history

Seller (Restaurant Administrator)

- Manages food items
- Uploads images
- Views orders
- Updates order status

The transaction process ensures that:

- Orders are only created after successful sandbox payment.
- Payment results are recorded.
- Email receipts are automatically sent.
- Role-based access is enforced.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: OrderFlow
Domain: Food E-commerce / Restaurant Transaction Management
Primary Users:

1. Buyers (Customers)
2. Sellers (Restaurant Administrators)

Problem Statement: Buyers and Sellers need a structured, secure platform to manage food ordering, inventory, and sandbox payment processing without exposing sensitive data.
Solution:A dual-role web and mobile system utilizing a Layered Architecture, featuring JWT authentication, Stripe Test Mode for secure transaction simulation, external Nutrition API integration, and automated SMTP email receipts.

2.2 Core User Journeys

Journey 1: Buyer Order & Sandbox Payment Flow

- User visits web or mobile application and creates a BUYER account.
- System sends SMTP Welcome Email.
- User browses food items, viewing fetched Nutrition API data (calories/nutrients).
- Adds items to cart and proceeds to checkout.
- Enters Stripe test card credentials (e.g., 4242...).
- Stripe verifies payment; system records successful payment result.
- System generates Order and sends an SMTP Order Receipt.

Journey 2: Seller Inventory & File Management

- User logs in with SELLER credentials.
- Navigates to the protected Seller Dashboard.
- Uploads a new food item, including uploading an image file.
- System stores the file on the server and saves the path in the database.
- Food item becomes immediately available for Buyers to purchase.

Journey 3: Role-Based Security Enforcement

- BUYER attempts to access the Seller Dashboard or food management API.
- System intercepts the request, validates the JWT role.
- System returns a `403 Forbidden` error, blocking access.

2.3 Feature List (MoSCoW)

MUST HAVE

- User authentication (Register, Login, Logout, `/me` endpoint).
- Google OAuth Login.
- Role-Based Access Control (BUYER vs. SELLER).
- Core Transaction System (Order, OrderItems, Payment).
- Stripe Test Mode integration (Sandbox payments).
- Android Mobile Application (Kotlin, XML, Retrofit).

SHOULD HAVE

- File Upload functionality for food images.
- External API Integration (Nutrition API).
- SMTP Email Sending (Welcome & Receipts).

COULD HAVE

- Order history tracking.

WON'T HAVE

- Real money payment processing (Production mode disabled).
- Live GPS delivery tracking.
- Multi-restaurant expansion.

2.4 Detailed Feature Specifications

Feature: Authentication & Security

- Screens: Registration, Login.
- Functions: Email/Password auth, Google OAuth integration.
- Security: JWT tokens, BCrypt password hashing, Strict DTO usage (no password exposure).

Feature: Role-Based Access Control

- API Restrictions: Only SELLER manages food. Only BUYER places orders.
- UI Restrictions: Dynamic UI rendering based on role; `403 Forbidden` for unauthorized API hits.

Feature: Core Transaction & Sandbox Payment

- Entities: Order, OrderItems, Payment.
- Process: Create Stripe Payment Intent → Process test card → Store payment result → Create Order *only* if successful.
- Validation: Atomic database transactions to prevent orphaned orders.

Feature: External Integrations

- Nutrition API: Fetches and displays real-time calorie/nutrient data on food details.
- SMTP Server: Triggers automated transactional emails.
- File Storage: Handles multipart file uploads for Seller images.

2.5 Acceptance Criteria

- **AC-1: Sandbox Payment Integrity**
    - *Given* I am a BUYER at checkout
    - *When* I process a payment using test card credentials
    - *Then* the system shall communicate with Stripe
    - *And* store the payment result in the database
    - *And* only generate the Order entity if the payment was successful.
- **AC-2: Secure Role Separation**
    - *Given* I am logged in with a BUYER token
    - *When* I attempt to trigger a SELLER-only API endpoint (e.g., add food item)

    - *Then* the system shall reject the request and return a `403 Forbidden` status.
- **AC-3: External API & File Upload**
    - *Given* I am viewing a food item in the application
    - *When* the item loads
    - *Then* the system shall display the uploaded image from the server file path
    - *And* display accurate nutritional data fetched from the external Nutrition API.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds.
- Payment processing complete: ≤ 5 seconds.
- Concurrency: Support 100 concurrent users.
- Database queries complete: ≤ 500ms.

3.2 Security Requirements

- HTTPS required for all communications.
- Password hashing with BCrypt.
- JWT token authentication.
- Role-based authorization for all endpoints.
- Strict DTO usage (no password exposure in JSON responses).
- SQL injection prevention.

3.3 Reliability Requirements

- Global exception handling implemented.
- Atomic database transactions (Order depends on Payment success).
- Unique Order IDs generated for tracking.
- Payment results permanently stored.

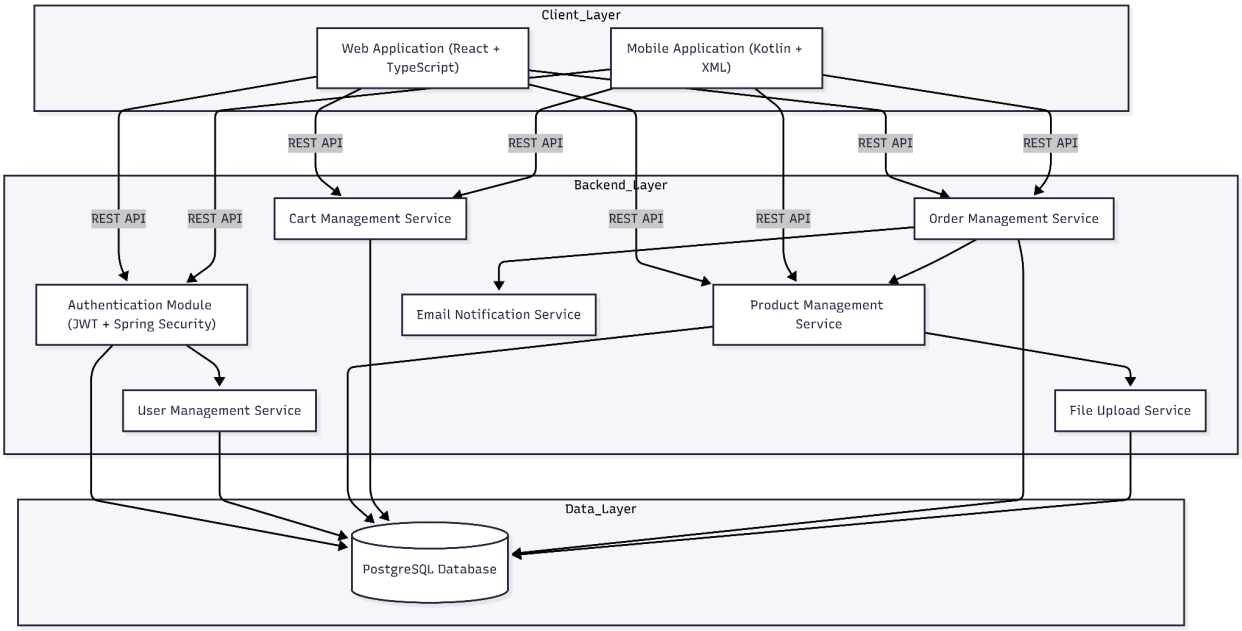
3.4 Database Requirements

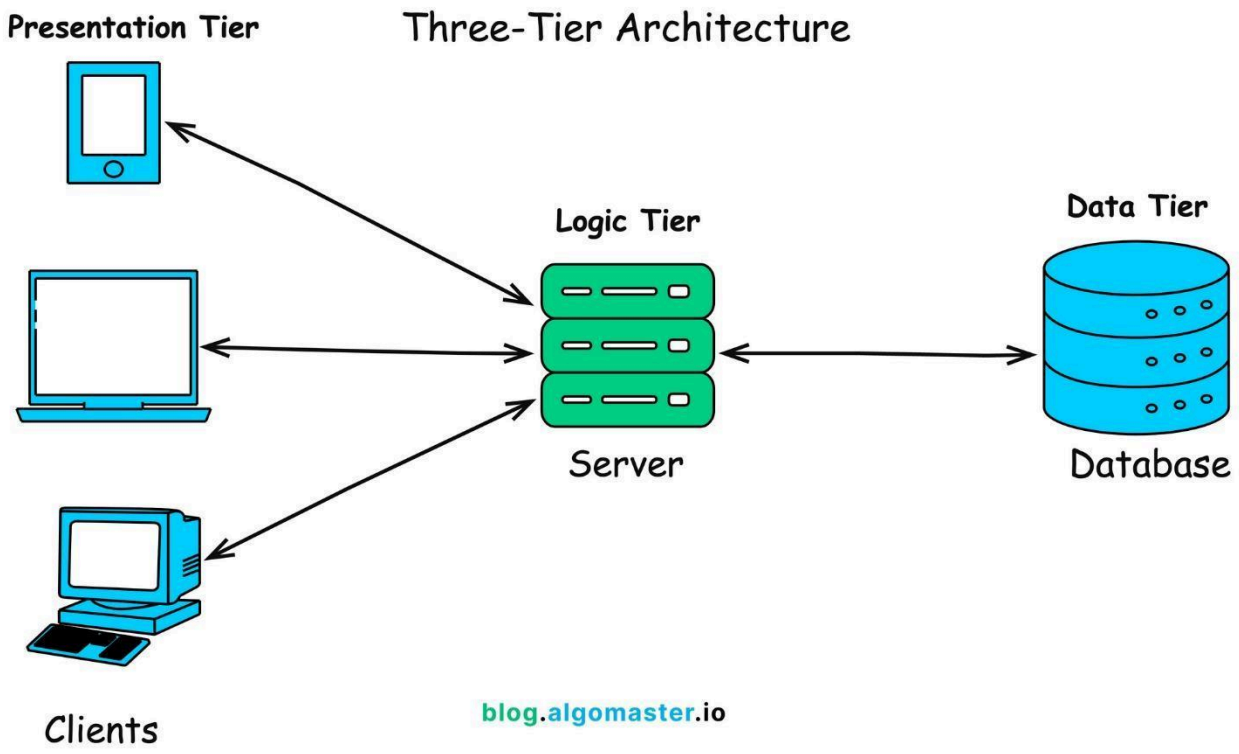
- **Required Tables:** `users`, `food_items`, `carts`, `cart_items`, `orders`, `order_items`, `payments`.

- **Architecture:** Proper relational constraints and foreign keys must be implemented to maintain data integrity.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram





Major Components

Client Layer

- Web Application (Buyer & Seller)
- Mobile Application (Buyer)

Backend Layer

- Authentication & Authorization Module (JWT, Role-Based Access)
- User Management Module
- Product Management Module (Seller)
- Cart Management Module (Buyer)
- Order Management Module (Transaction Processing)
- File Upload Module (Product images)
- Email Notification Module (SMTP)

Data Layer

- PostgreSQL Database
- JPA/Hibernate ORM

Component Interaction

- Clients communicate with the backend via RESTful APIs.
- Backend services interact through service and repository layers.
- JWT secures communication between client and server.
- Database handles persistent storage.

Technology Stack

Backend:
Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA

Database:
PostgreSQL 14+

Web Frontend:
React 18, TypeScript, Tailwind CSS, Axios

Mobile:
Kotlin, **XML-based UI**, Retrofit, Room

Build Tools:
Maven (Backend), npm/yarn (Web), Gradle (Android)

Deployment:
Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Registers a new Buyer or Seller account.
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "BUYER | SELLER"
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"accessToken": "<token>",
	"refreshToken": "<token>"
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

User Login

Description	Authenticates user and issues JWT tokens.
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses

Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>"
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"accessToken": "<token>",
	"refreshToken": "<token>"
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

5.3 Error Handling

HTTP Status Codes

- **200 OK – Successful request**
- **201 Created – Resource created**
- **400 Bad Request – Invalid input**
  401 Unauthorized – Authentication failed
- **403 Forbidden – Insufficient permissions**
  404 Not Found – Resource not found
- **409 Conflict – Duplicate resource**
- **500 Internal Server Error – Server error**

Common Error Codes

- **AUTH-001: Invalid credentials**
- **AUTH-002: Token expired**
- **AUTH-003: Insufficient permissions**
- **VALID-001: Validation failed**
- **DB-001: Resource not found**
- **DB-002: Duplicate entry**
  BUSINESS-001: Insufficient stock
- **SYSTEM-001: Internal server error**

Error Code Examples

Example 1	json
	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

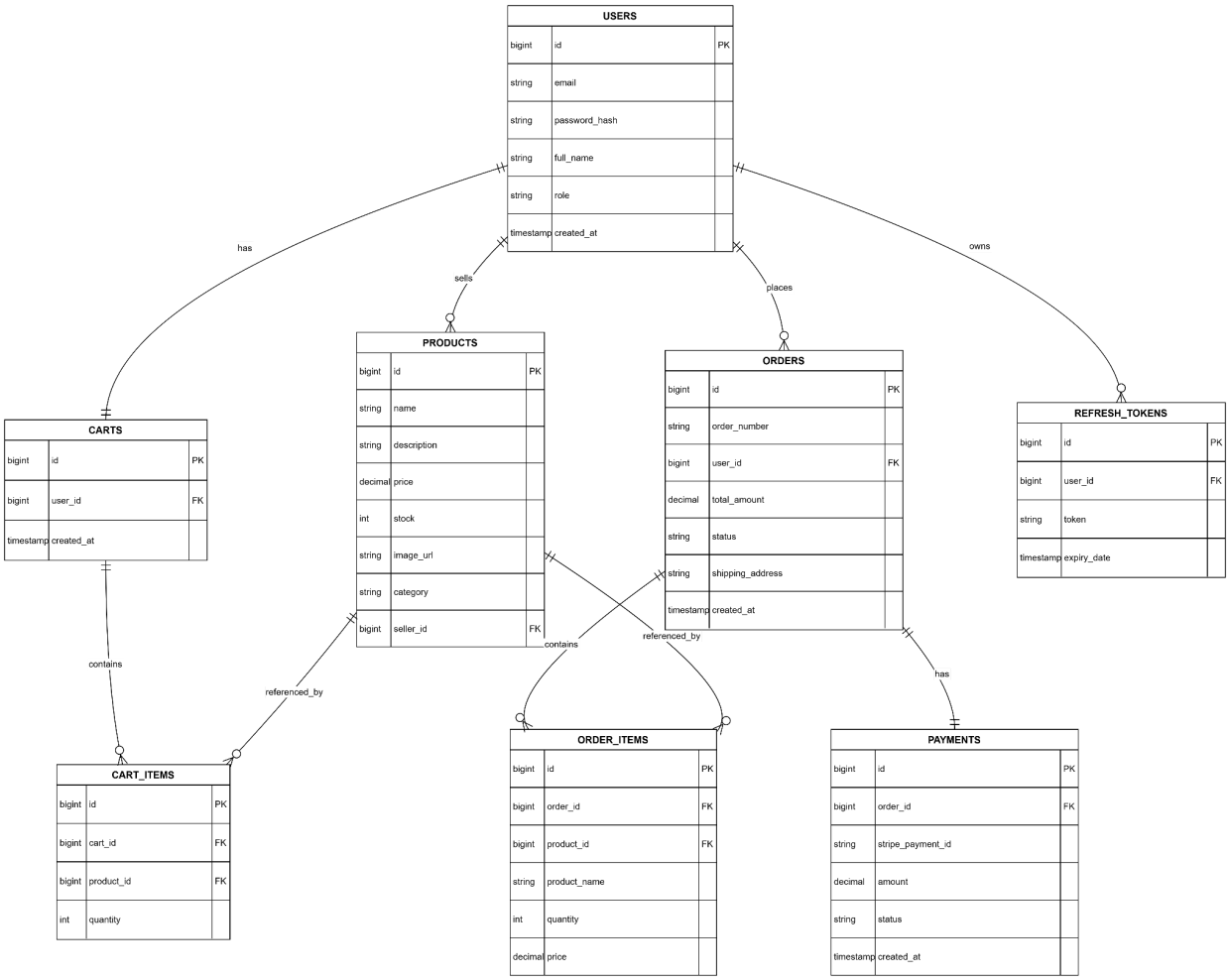
Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Common Error Codes

- **AUTH-001: Invalid credentials**
- **AUTH-002: Token expired**
- **AUTH-003: Insufficient permissions**
- **VALID-001: Validation failed**
- **DB-001: Resource not found**
- **DB-002: Duplicate entry**
  BUSINESS-001: Insufficient stock
- **SYSTEM-001: Internal server error**

6.0 DATABASE DESIGN

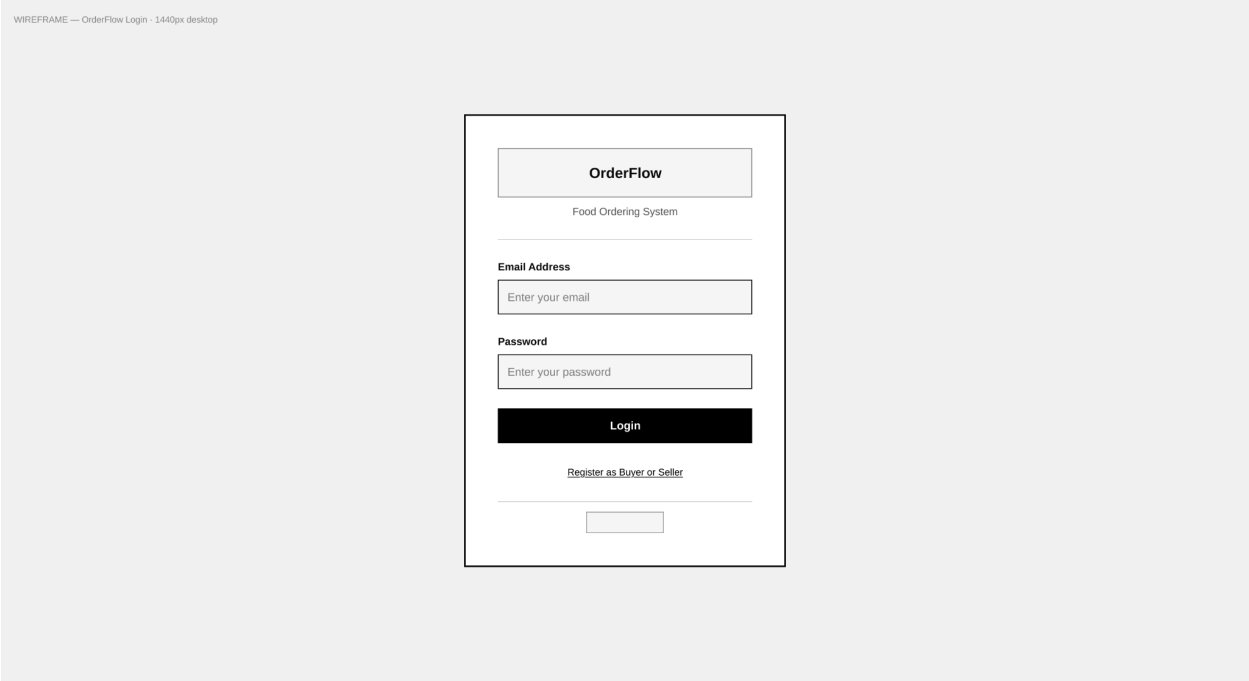
6.1 Entity Relationship Diagram



7.0 UI/UX DESIGN

7.1 Web Application Wireframes

https://www.figma.com/design/Of74uXdke8vbgBqL6XU7Yc/OrderFlow?node-id=0-1&m=dev&t=WObiq4JWHdsBAlVc-1



OrderFlow

Search meals or restaurants

Cart | Profile

WIREFRAME — Buyer Homepage · Food Marketplace

All | Pizza | Burgers | Rice Meals | Drinks | Desserts

Available Meals 9 items

Meal Image

Margherita Pizza
Pizza Palace
₱250
Add to Cart

Meal Image

Double Cheeseburger
Burger Town
₱180
Add to Cart

Meal Image

Adobo Rice Bowl
Casa Filipino
₱120
Add to Cart

Meal Image

BBQ Bacon Burger
Burger Town
₱210
Add to Cart

Meal Image

Pepperoni Pizza
Pizza Palace
₱280
Add to Cart

Meal Image

Mango Shake
Fresh Blends
₱85
Add to Cart

Meal Image

Sinigang Set
Casa Filipino
₱150
Add to Cart

Meal Image

Chocolate Sundae
Sweet Corner
₱95
Add to Cart

Meal Image

Iced Americano
Coffee Hub
₱75
Add to Cart

WIREFRAME — Meal Detail Page · ID: 2

‹ Back to Marketplace

OrderFlow | Search meals or restaurants | Cart | Profile ▾

Pizza

Meal Image (500×500)

RESTAURANT Pizza Palace

₱250.00

DESCRIPTION

Classic Margherita pizza with fresh mozzarella, ripe tomatoes, and fragrant basil leaves on a crispy thin crust. Made fresh with our homemade tomato sauce and premium ingredients.

QUANTITY

− 1 +

Add to Cart

Related Meals

Double Cheeseburger	Adobo Rice Bowl	BBQ Bacon Burger	Pepperoni Pizza	Mango Shake
₱180	₱120	₱210	₱280	₱85

OrderFlow | Search meals or restaurants | Cart | Profile ▾

WIREFRAME — Cart Page

‹ Continue Shopping

Your Cart

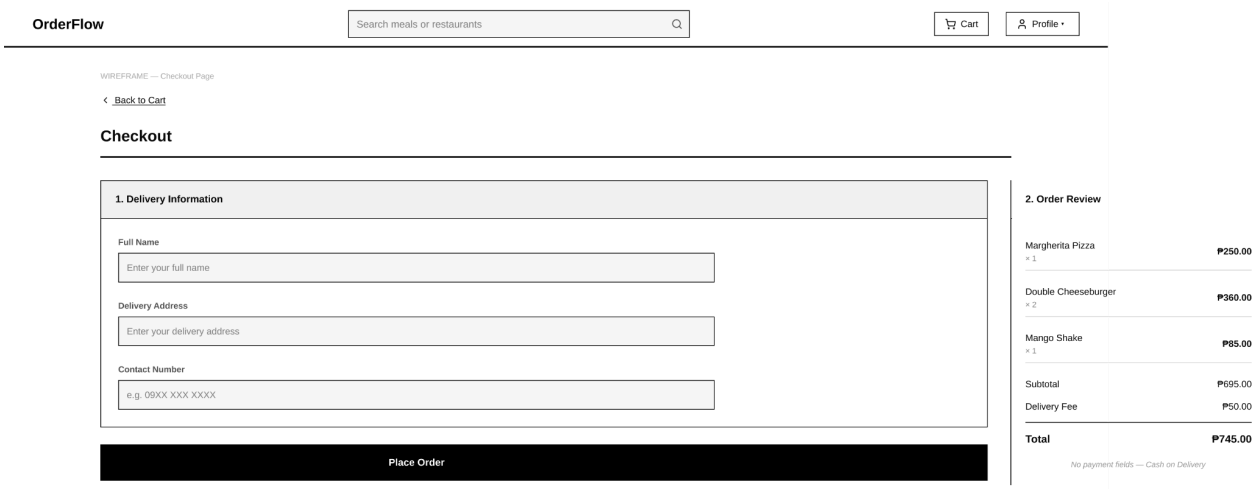
3 item(s)

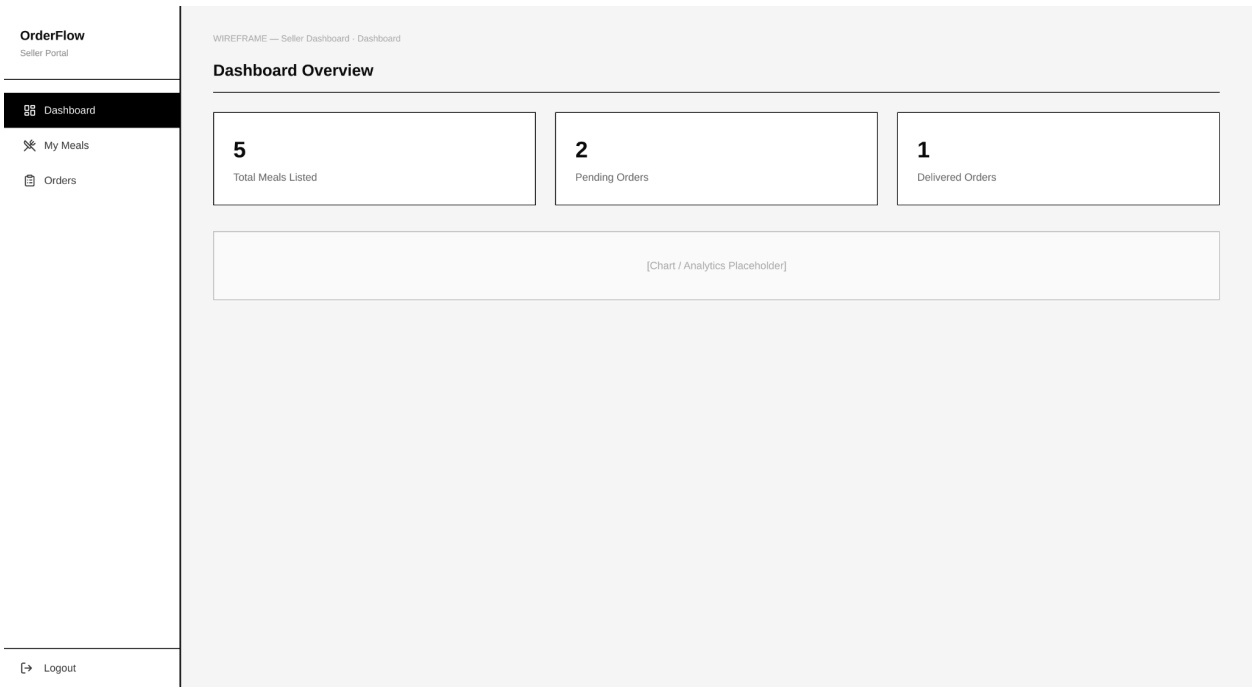
IMG	MEAL NAME	RESTAURANT	QUANTITY	PRICE	
	Margherita Pizza	Pizza Palace	− 1 +	**₱250.00**	×
	Double Cheeseburger	Burger Town	− 2 +	**₱360.00**	×
	Mango Shake	Fresh Blends	− 1 +	**₱85.00**	×

Order Summary

Subtotal	₱695.00
Delivery Fee	₱50.00
Total	**₱745.00**

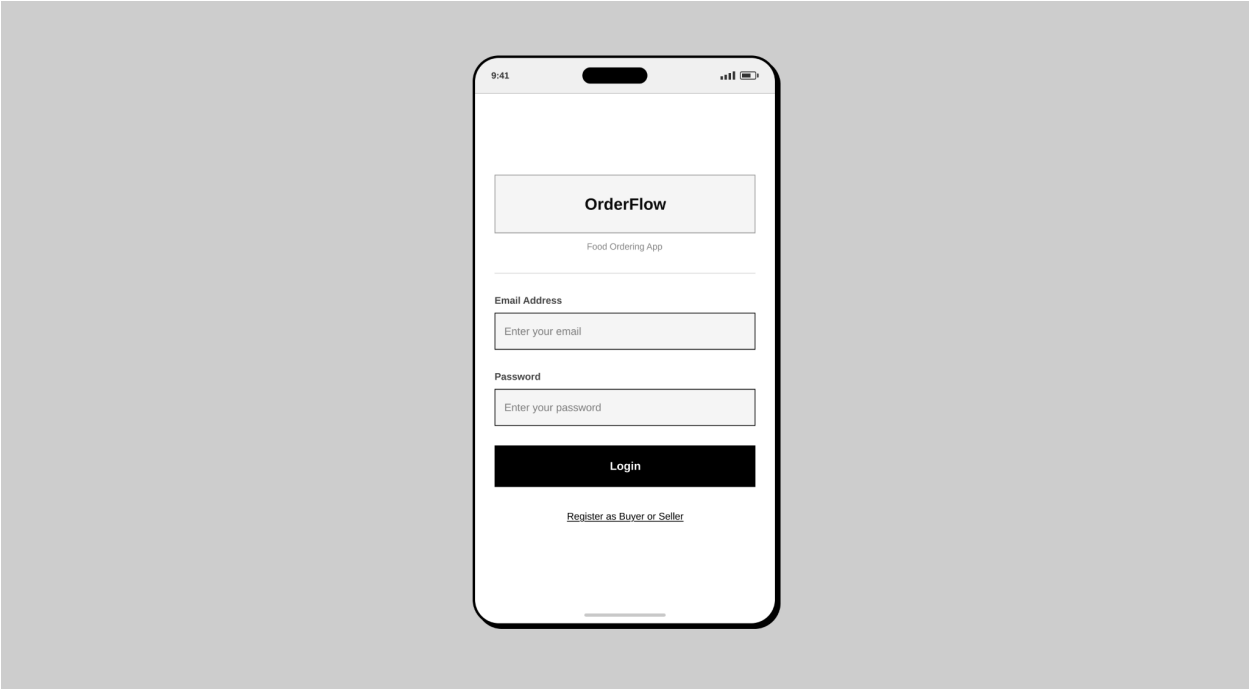
Proceed to Checkout

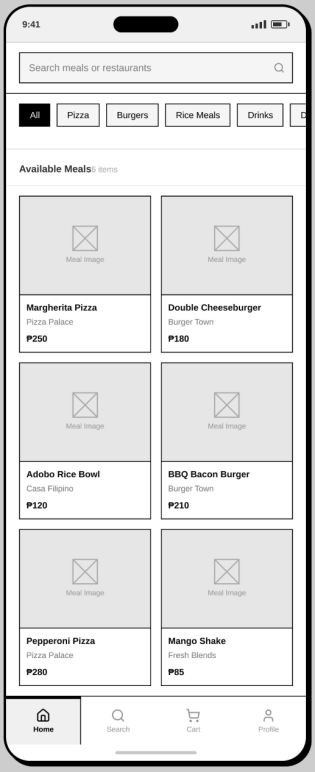

OrderFlow
Search meals or restaurants
Cart
Profile
WIREFRAME — Checkout Page
Back to Cart
Checkout
1. Delivery Information
Full Name
Enter your full name
Delivery Address
Enter your delivery address
Contact Number
e.g. 09XX XXX XXXX
Place Order
2. Order Review
Margherita Pizza
× 1
₱250.00
Double Cheeseburger
× 2
₱360.00
Mango Shake
× 1
₱85.00
Subtotal
₱695.00
Delivery Fee
₱50.00
Total
₱745.00
No payment fields — Cash on Delivery

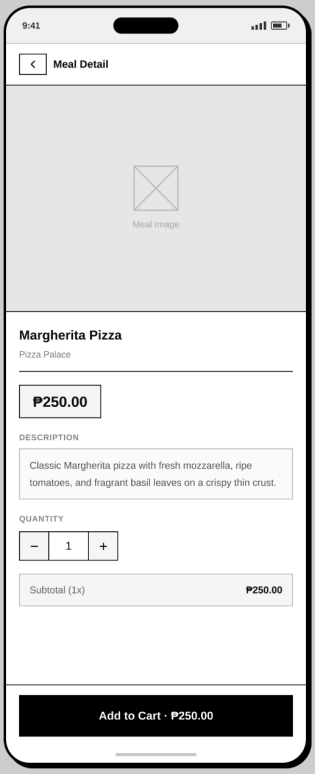

OrderFlow
Seller Portal
Dashboard
My Meals
Orders
Logout
WIREFRAME — Seller Dashboard · Dashboard
Dashboard Overview
5
Total Meals Listed
2
Pending Orders
1
Delivered Orders
[Chart / Analytics Placeholder]

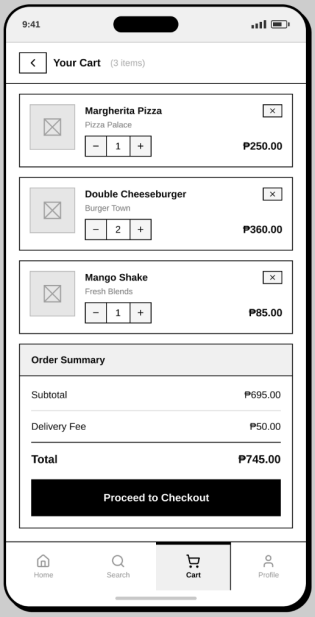
7.2 Mobile Application Wireframes

https://www.figma.com/design/Of74uXdke8vbgBqL6XU7Yc/OrderFlow?node-id=2-666&t=WObiq4JWHdsBAlVc-1




9:41
Search meals or restaurants
All
Pizza
Burgers
Rice Meals
Drinks
Available Meals6 items
Meal Image
Margherita Pizza
Pizza Palace
₱250
Meal Image
Double Cheeseburger
Burger Town
₱180
Meal Image
Adobo Rice Bowl
Casa Filipino
₱120
Meal Image
BBQ Bacon Burger
Burger Town
₱210
Meal Image
Pepperoni Pizza
Pizza Palace
₱280
Meal Image
Mango Shake
Fresh Blends
₱85
Home
Search
Cart
Profile


9:41
Meal Detail
Meal Image
Margherita Pizza
Pizza Palace
₱250.00
DESCRIPTION
Classic Margherita pizza with fresh mozzarella, ripe tomatoes, and fragrant basil leaves on a crispy thin crust.
QUANTITY
− 1 +
Subtotal (1x)
₱250.00
Add to Cart · ₱250.00


9:41
Your Cart (3 items)
Margherita Pizza
Pizza Palace
− 1 +
₱250.00
Double Cheeseburger
Burger Town
− 2 +
₱360.00
Mango Shake
Fresh Blends
− 1 +
₱85.00
Order Summary
Subtotal
₱695.00
Delivery Fee
₱50.00
Total
₱745.00
Proceed to Checkout
Home
Search
Cart
Profile

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

- Day 1-2: Project setup and documentation
- Day 3-4: Complete FRS and NFR
- Day 5-7: System architecture and diagram design

Week 2: Detailed Design

- Day 1-2: API specification (Buyer & Seller endpoints)
- Day 3-4: Database design and ERD
- Day 5-6: Web and Mobile wireframes
- Day 7: Finalization of implementation plan

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

- Day 1: Spring Boot setup and dependencies
- Day 2: PostgreSQL configuration and entity setup
- Day 3: JWT authentication implementation
- Day 4: User role management (Buyer/Seller)
- Day 5: Product CRUD (Seller functionality)

Week 4: Core Features

- Day 1: Cart functionality (Buyer)
- Day 2: Order processing logic (Checkout without payment)
- Day 3: Search and filtering (Food categories)
- Day 4: Validation and error handling
- Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

- Day 1: React setup with TypeScript
- Day 2: Authentication pages (Login/Register with role selection)
- Day 3: Food product listing page
- Day 4: Meal detail page
  Day 5: Shopping cart implementation

Week 6: Complete Web Features

- Day 1: Checkout and order confirmation flow
- Day 2: Buyer order history
- Day 3: Seller dashboard (Meal & Order management)
- Day 4: Responsive UI polish
- Day 5: API integration testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

- Day 1: Android project setup
- Day 2: Authentication screens
- Day 3: Food browsing screen

- Day 4: Shopping cart screen
- Day 5: Retrofit API integration

Week 8: Complete Mobile App

- Day 1: Checkout flow (No payment integration)
- Day 2: Buyer order history and tracking
- Day 3: UI polish and basic animations
- Day 4: Emulator and device testing
- Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

- Day 1: End-to-end testing (Web + Mobile)
- Day 2: Bug fixes
- Day 3: Security review (JWT & role validation)
- Day 4: Performance optimization
- Day 5: Documentation update

Week 10: Deployment

- Day 1: Backend deployment
- Day 2: Web deployment
- Day 3: Mobile APK release
- Day 4: Final testing
- Day 5: Final submission

Milestones

- M1 (Week 2): Design documents complete
- M2 (Week 4): Backend API functional
- M3 (Week 6): Web app complete
- M4 (Week 8): Mobile app complete
- M5 (Week 10): Fully integrated system

Critical Path

- JWT authentication
- Role-based authorization
- Product CRUD
- Cart logic
- Order processing
- Cross-platform integration